CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

What this is

A personal indoor-cycling training log (Christopeit AX 4000 + Kinomap), not a software project. No build system, no tests, no package manager — just static self-contained HTML dashboards, CSV exports, and a SQL dump, all manually maintained snapshots of the same underlying training sessions.

Architecture: data flows one way, by hand

There is no shared data source and no fetch/AJAX/CSV-loading anywhere. Every `.html` file embeds its own hard-coded JS data array in a `<script>` block and renders it with Chart.js (loaded from `cdnjs.cloudflare.com`, no local deps). Updating one file does **not** update the others — each is a point-in-time export that must be re-synced manually when new sessions are added.

- **`training.html`** — the most current and most complete file. Contains two separate arrays that must both be updated when adding a session:
  - `const raw=[...]` (~line 233): `[date, watt, kcal, hf, week]` — compact per-session data used for the chart aggregations (daily/weekly rollups).
  - `const rows=[...]` (~line 505): the detailed table, one object per session (`{n, w, tag, dat, dur, k, kpm, watt, hf, kum, rec, rr, rr_ruhe, rr_abend, note}`). The `note` field is the only place route name, distance, elevation, temperature, and cadence live (as a free-text `·`-separated string prefixed `Kinomap · ...` for Kinomap rides) — there's no structured field for these.
  - Rest days appear in `rows` with `n:'–'` and null values; skip them when extracting session data.
  - **Adding a session touches more than `raw`/`rows`.** The daily/weekly charts and the hero metric bar do *not* derive from `raw`/`rows` at render time — they read separate hardcoded arrays/values that must be updated by hand in lockstep:
    - Hero `.metric-val` divs (~line 76-81): Einheiten, Kalorien total, Dauer total, Ø kcal/min, Ø Watt, Letzte Einheit.
    - `wkKcal`/`wkMin`/`wkWatt`/`wkKpm` (~line 449): per-week rollups used by the Wöchentlich-tab charts.
    - The `'Erreicht'` dataset in the `cPlan` chart (~line 490): a *third* copy of the per-week Ø-Watt series, used by the Plan-200W tab.
    - `endDate` in `parseDate('25.5'),endDate=parseDate(...)` (~line 408): must be bumped to the newest session date or the daily chart's date range won't include it.
    - `getWeek()` (~line 390) and the `wn` map (~line 612): week boundaries are hardcoded 7-day blocks: bump these (and `wkLabels` ~line 450) when a new week starts, closing the previous week's range and opening the new one — don't leave the latest week open-ended once a session lands in the next one. Forgetting one of these is exactly what produced a wrong weekly total after adding a session — always grep the new date/week number across the whole file after editing `raw`/`rows`, don't assume the rest is derived. Line numbers drift as the file changes — `grep -n` the anchor strings above rather than trusting these numbers blindly.
- **`training_dashboard.html`** — an older snapshot with the same `raw`-array shape as `training.html` but fewer rows (stale, not kept in sync).
- **`training_data.csv`** — flat export of `training.html`'s `raw` array (`Nr,Tag,Datum,Dauer_min,kcal,kcal_min,Watt`). `Tag` (weekday) and `Dauer_min`/`kcal_min` are derived, not stored in the source — see formula below.
- **`kinomap_rennwerte.csv`** — Kinomap-only sessions (the subset of `rows` whose `note` starts with `Kinomap`), parsed out of the `note` free text into structured columns: `Datum,Tag,Strecke,Distanz_km,Dauer_min,kcal,kcal_mi`. Not every session is a Kinomap ride, so this has fewer rows than `training_data.csv` for the same dates.
- **`training_data_mysql.sql`** — an even older, more stale export with extra fields (blood pressure `rr` values, cumulative kcal) not present in the CSVs.
- **`stufentest.html` / `stufentest_regression.html`** — standalone step-test (Stufentest) watt/HR calibration charts, unrelated data (`watts[]`/`bpms[]` arrays), used to derive the HR-based watt estimation formula.
- **`wkg_histogram.html`**, **`training_slideshow.html`** — other standalone single-purpose dashboards with their own embedded data.

- **`PulsCharts/`** — a self-contained folder: `puls_graphen_uebersicht.html` plus the watch-screenshot `.jpg` files it displays, referenced by plain relative filename (no subfolder) in the `<img src>`.

Generating training.pdf

`training.pdf` is a print export of `training.html` (all four tabs, light theme, landscape, pure white background) and must be regenerated by hand after editing `training.html` — nothing does this automatically. training.html's CSS/JS already has the print-specific fixes baked in (see below); the generation script only needs to force all tabs visible and force light mode, since Chrome's `--blink-settings=preferredColorScheme` flag does not reliably override the page's own `matchMedia('(prefers-color-scheme: dark)')` check.

Recipe (uses headless Google Chrome + poppler's `pdftoppm`/`pdftotext` for verification):

```
SCRATCH=/path/to/scratch   # any writable temp dir
python3 - <<'EOF'
p = '/Users/haraldbeker/training/training.html'
s = open(p).read()
s = s.replace(".sec{display:none}", ".sec{display:block}")   # show all 4 tabs
s = s.replace("const isDark=matchMedia('(prefers-color-scheme: dark)').matches;", "const isDark=f
open('/path/to/scratch/training_print.html', 'w').write(s)
EOF

"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" \
  --headless --disable-gpu --no-pdf-header-footer --virtual-time-budget=8000 \
  --print-to-pdf="$SCRATCH/final.pdf" "file://$SCRATCH/training_print.html"

cp "$SCRATCH/final.pdf" /Users/haraldbeker/training/training.pdf
```

Before trusting the output, verify at high DPI — the low-res inline PDF preview is not reliable enough to catch any of the bugs below:

```
pdftoppm -png -r 200 -f 1 -l 1 /Users/haraldbeker/training/training.pdf "$SCRATCH/check"
# crop/zoom the daily "Ø Watt pro Tag" chart and confirm the tallest bar's pixel
# height actually lines up with its real value's gridline (see below — this has
# been silently wrong multiple times even when axis labels looked fine).
```

Pitfalls already fixed in training.html — don't reintroduce them:

- **Chart bars/lines can render squashed relative to their own y-axis even when Chart.js's config and data are correct** (e.g. a value of 213 only reaching the "140" gridline on a 100–230 axis). Root cause: Chart.js's default ~1000ms bar-growth animation gets captured mid-animation by the headless PDF snapshot, since `--virtual-time-budget` doesn't reliably advance `requestAnimationFrame`. Fixed by `animation:false` in `baseOpts` (~line 428) — every chart spreads `...baseOpts`, so this covers all 8 charts. If squashing ever comes back, check this line first; a `beforeprint` chart-resize listener and stripping stale canvas `width`/`height` attributes were both tried and were **not** the actual fix, despite looking related.
- **Hardcoded y-axis `min`/`max` per chart** (the `by(label,min,max)` helper, used for cWatt/cKcal/cTime/cPuls/cWeekPerf/cPl are NOT auto-derived from the data, so adding sessions can push real values past an old max and clip bars flat at the top. Recompute actual min/max from `raw`/`rows` before trusting an export (this already happened once: the kcal chart's max of 700 was clipping real days up to 841, fixed by bumping to 900).
- **Print-only CSS overrides** live in `@media print{...}` near line 23 (pure white `--bg`/`--surface`/`--surface2`, `@page{size:landscape}`, `body{max-width:none}` so the wide session table fits) and near line 51 (`.rec-row td{background:none}` — suppresses the "sehr gute Leistung" row highlight in print to save toner; must be declared *after* the base `.rec-row td` rule, since two rules with equal CSS specificity resolve by source order even across separate `@media print` blocks).
- `@media print{.card{break-inside:avoid;page-break-inside:avoid}}` (next to `.card{...}`, ~line 36) stops a chart card from being sliced across a page boundary.

Key domain facts

- Watt is estimated from kcal/min, not measured directly: `Watt = (kcal/min × 1000 × 4.185) / 60 × 0.25`. Inverting it: `Dauer_min = kcal × 17.4375 / Watt`.
- Dates are `D.M` with no year (implicit 2026), no leading zeros (e.g. `'5.6'`, `'14.8'`).
- Training weeks are fixed calendar boundaries starting Monday-ish from 25.5 (`getWeek()` / `wn` map in `training.html`), not ISO weeks — check the `wn` object (~line 612 of `training.html`) before assuming a date's week number.
- A day can have multiple sessions (e.g. two rides logged the same date); when aggregating per day, sum kcal and average watt across that day's entries (see `dayMap` in `training.html`).